Machine learning Tek 5: classification, logistic regression

Content

General classification problem

Classification : supervised learning in which we predict a **class** (the set of possible labels is discrete (most often **finite**)).

- $\mathcal{X} = \mathbb{R}^d$
- $\mathcal{Y} = \{-1, 1\}$ or $\mathcal{Y} = \{0, 1\}$ for **binary** classification (2 possible classes).
- $l(y, z) = 1_{y \neq z}$ ("0-1" loss)
  `https://en.wikipedia.org/wiki/Indicator_function`
- $F = \mathcal{Y}^{\mathcal{X}}$
- Examples : MNIST, ImageNet

Empirical risk

We learn from a dataset D_n of n **labeled examples**;

$$D_n = \{(x_i, y_i), i \in [1, n]\} \tag{1}$$

If the dataset is split into a train and a test set, and with the "0-1" loss, the train error writes

$$\frac{1}{n_{\text{train}}} \sum_{(x_i, y_i) \in (X_{train} \times y_{train})} 1_{\tilde{f}(x_i) \neq y_i} \tag{2}$$

Exercice 1 : What is a natural interpretation of the train error with the "0-1" loss ?
The **accuracy** is $1 -$ train error.

Let us compare the classification train error to the squared-loss train error that we studied last week :

- classification with "0-1" loss :

$$\frac{1}{n_{\text{train}}} \sum_{(x_i,y_i)\in(X_{train}\times y_{train})} 1_{\tilde{f}(x_i)\neq y_i} \tag{3}$$

- regression with squared-loss :

$$\frac{1}{n_{\text{train}}} \sum_{(x_i,y_i)\in(X_{train}\times y_{train})} (\tilde{f}(x_i)-y_i)^2 \tag{4}$$

We have not yet focused much on the optimization side of ML. But to find a good $\tilde{f}$, we have discussed **empirical risk minimization**, which consists in finding $\tilde{f}$ such that the train error is as small as possible, while avoiding overfitting at the same time.
As we will see next week, the possibility to differentiate the train error with respect to $\tilde{f}$ is one of the building blocks of the optimization process.

Let us compare the classification train error to the squared-loss train error that we studied last week :

- classification with "0-1" loss :

$$\frac{1}{n_{\text{train}}} \sum_{(x_i,y_i)\in(X_{train}\times y_{train})} 1_{\tilde{f}(x_i)\neq y_i} \tag{5}$$

- regression with squared-loss :

$$\frac{1}{n_{\text{train}}} \sum_{(x_i,y_i)\in(X_{train}\times y_{train})} (\tilde{f}(x_i)-y_i)^2 \tag{6}$$

The issue is that differentiating the train error with respect to the "0-1" loss is **not informative** ! The derivative is 0 almost everywhere.

Real-valued function

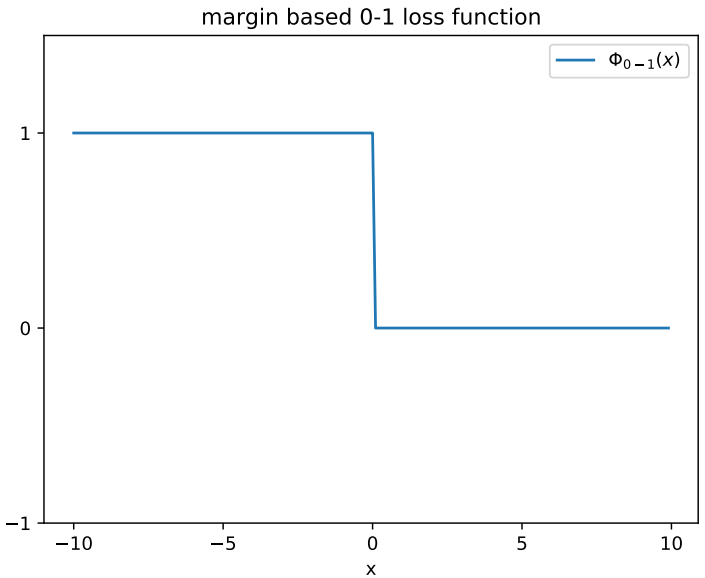
Instead of an application that predicts -1 or 1, we will learn $g : \mathcal{X} \to \mathbb{R}$ and define $f(x) = \text{sign}(g(x))$ with

$$\text{sign}(x) = \begin{cases} 1 \text{ if } x \geq 0 \\ -1 \text{ if } x < 0 \end{cases}$$

Empirical risk minimization

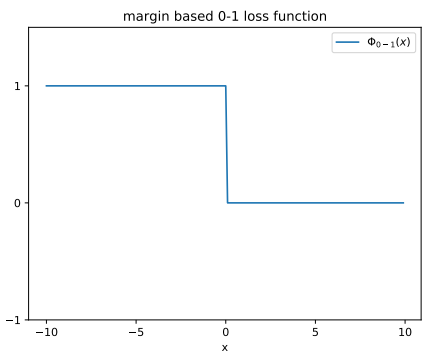
The corresponding empirical risk writes (Φ_{0-1} is defined by the image) :

$$\frac{1}{n}\sum_{i=1}^{n}\Phi_{0-1}(y_i g(x_i)) \tag{7}$$



Convex surrogate

Key idea : replace Φ_{0-1} by another function Φ that is easier to optimize (convex, differentiable) but still represents the correctness of the classification.
Natural question : but does minimizing the Φ-risk lead to a good "0-1" loss prediction ? Often yes, but answering this question in detail requires an advanced study.



Most common convex surrogates

Définition

Logistic loss

$$\Phi(u) = \log(1 + e^{-u}) \tag{8}$$

With linear predictors ($g(x_i) = \langle \theta, x_i \rangle$), this loss will lead to **logistic regression** (which is classification despite its name).

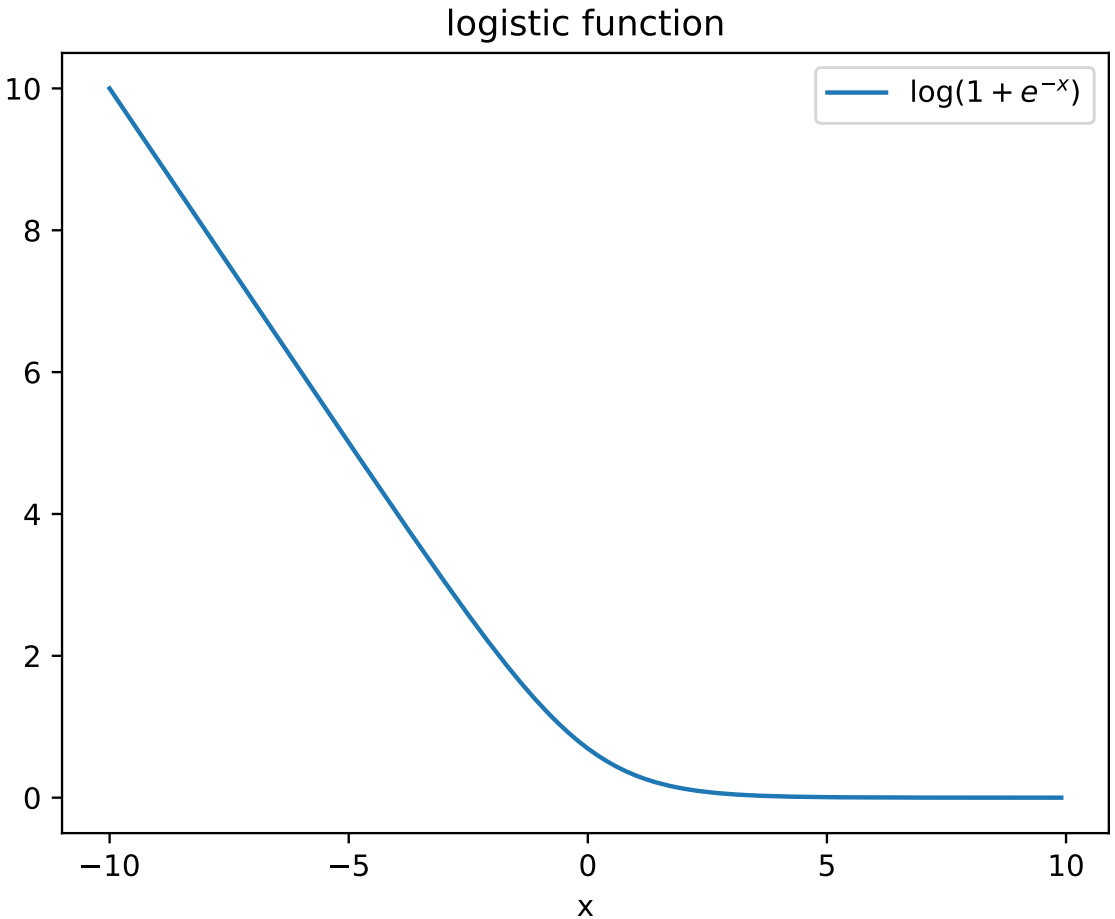
Most common convex surrogates

If $\mathcal{Y} = \{0, 1\}$, $\hat{y}$ is the prediction and y is the correct label, then we sometimes write :

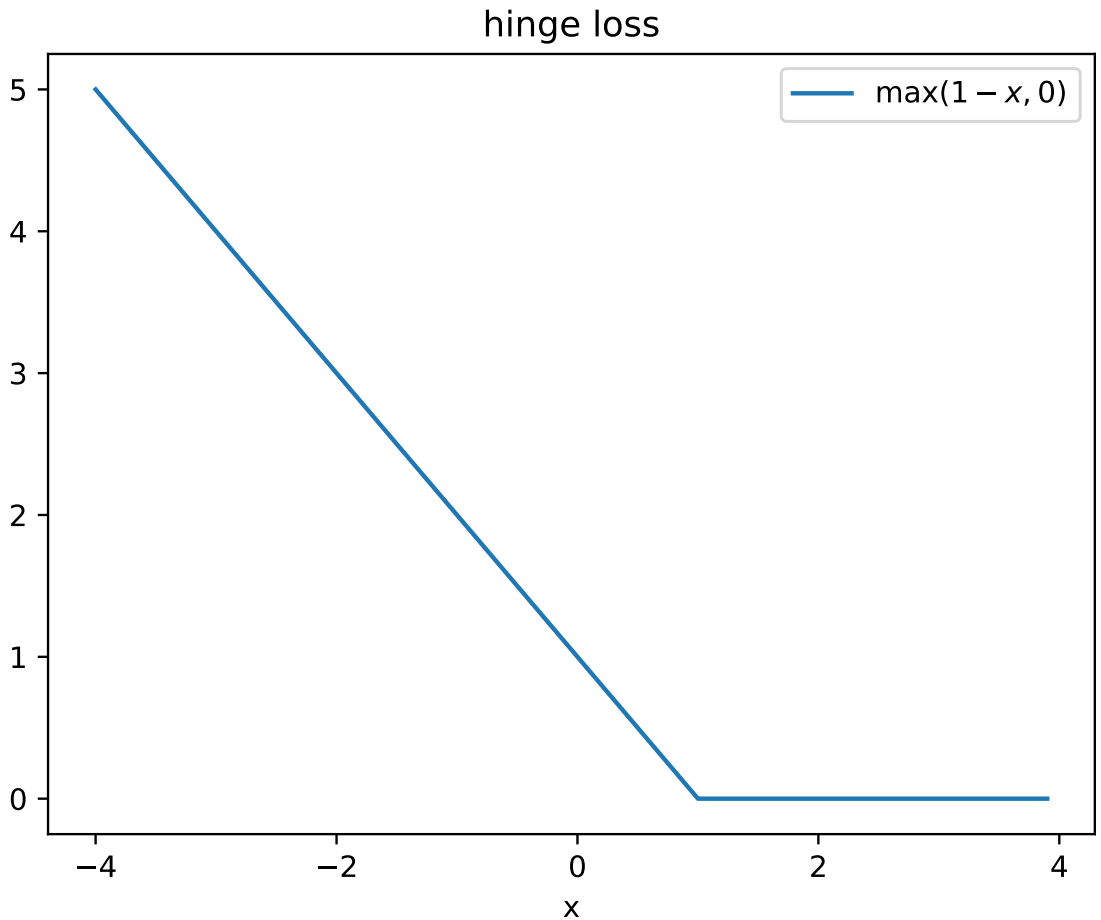
$$l(\hat{y}, y) = y \log(1 + e^{-\hat{y}}) + (1 - y) \log(1 + e^{\hat{y}}) \tag{9}$$

(cross entropy loss)

Logistic function



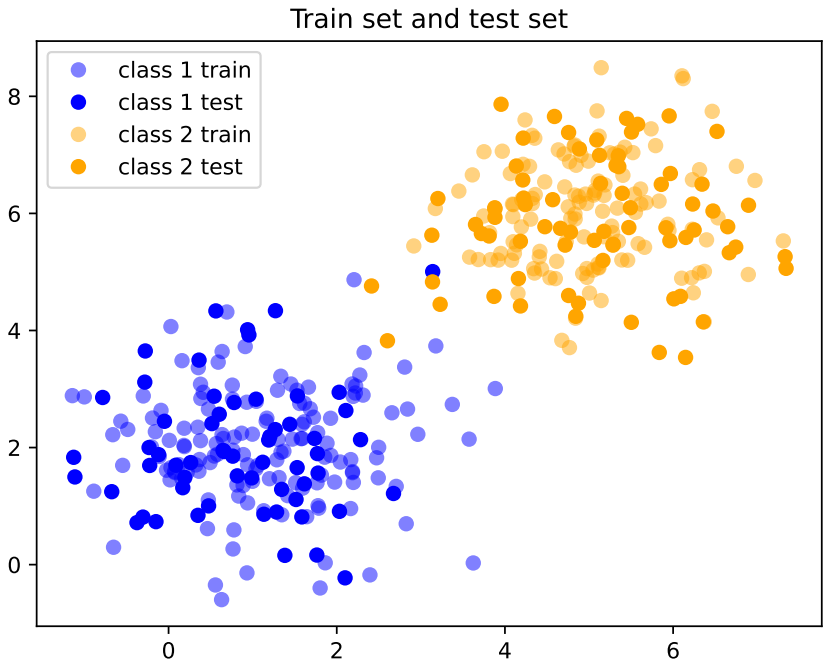
Hinge loss



Logistic regression

- $g(x) = \langle x, \theta \rangle = x^T \theta$.
- $f(x) = \text{sign}(\langle x^T \theta \rangle)$
- It can be seen as "linear regression applied to classification".

Application to a toy dataset



cd **code/day_2/1_logistic_regression/toy_dataset/**. We can use **logistic_regression.py** as a first example.

We can see that there are some mistakes on the test set.

No closed-form solution

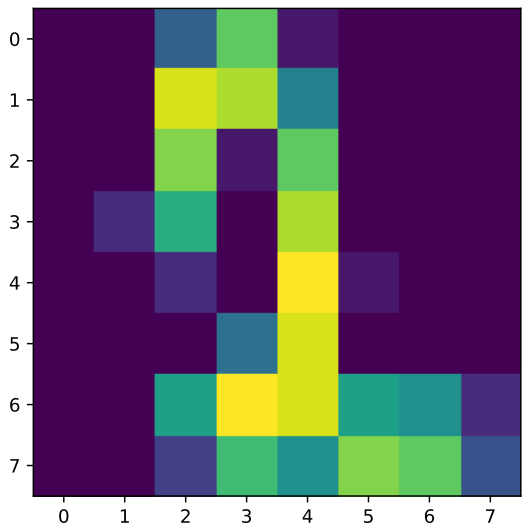
Since the loss is convex, to minimize it is sufficient to look for the cancellation of the gradient. However, the corresponding equation has no closed-form solution.
We thus need to use iterative algorithms (Gradient descent, Newton's method)
It is common practice to :

- regularize the logistic loss to avoid overfitting, for instance with a $L2$ penalty (as in ridge regression)
- use feature maps and classify with $\phi(x)$ instead of x.

Application to the digits dataset.

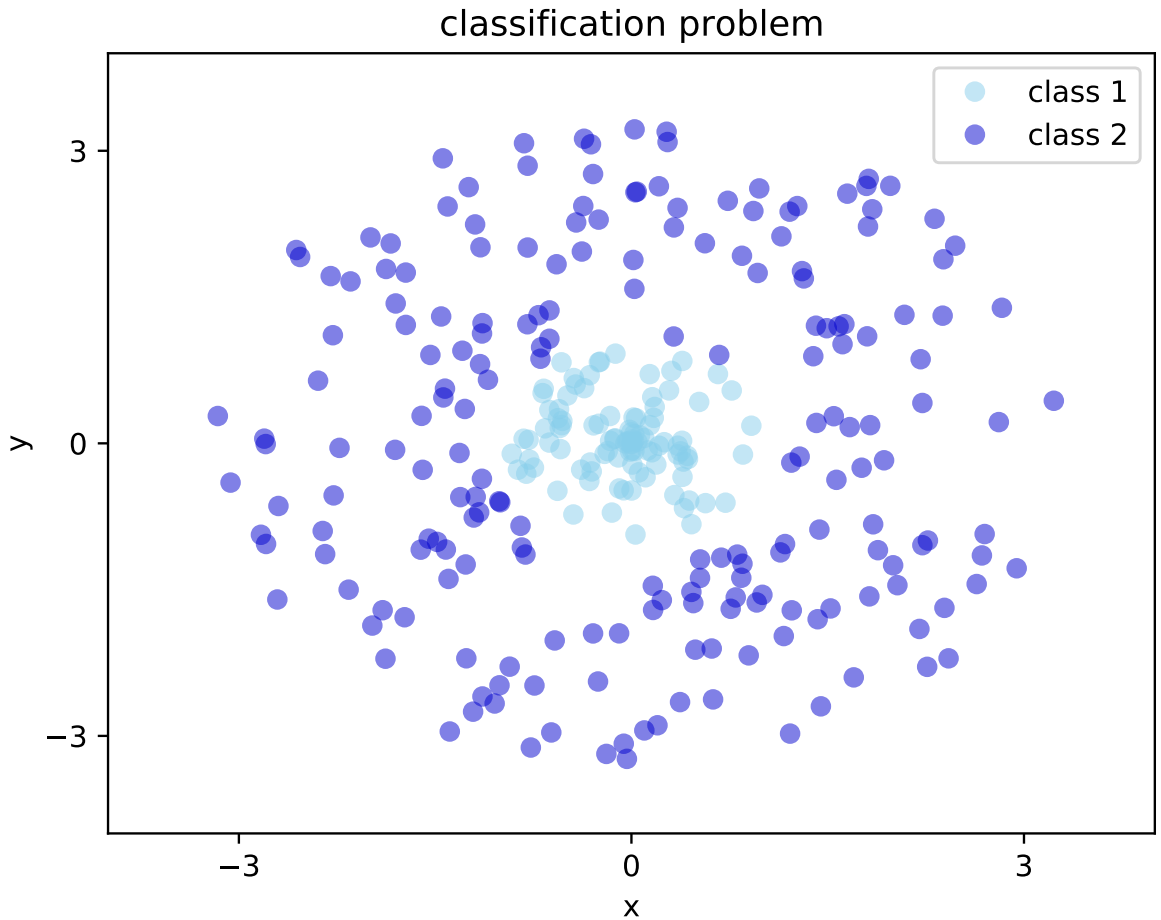
Exercice 2 :

cd **code/day_2/1_logistic_regression/digits_dataset/** and use **logistic_regression_digits.py** to perform a logistic regression on the **digits** dataset. Isn't there something weird ?

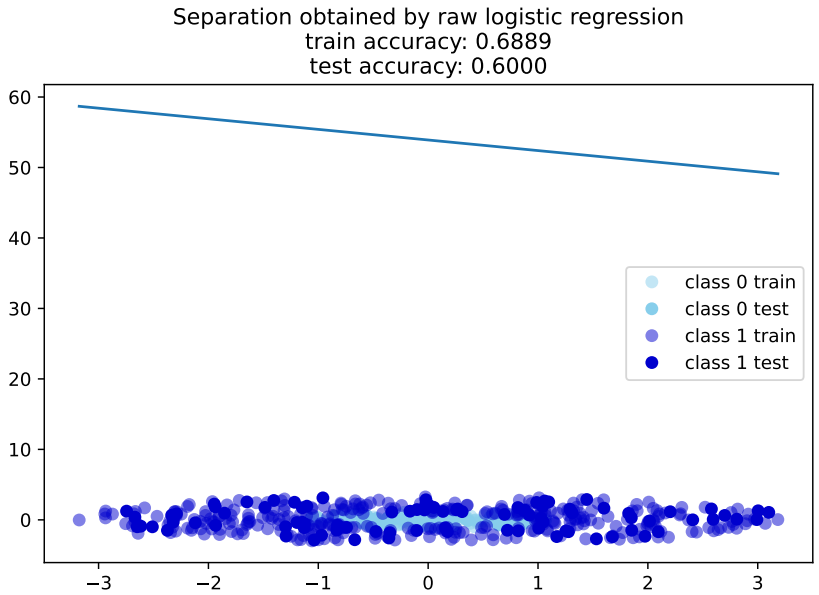


`https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html`

Exercice 3 : Can we use a linear classifier on this dataset ? What train and test accuracy do we expect ?



Vanilla logistic regression

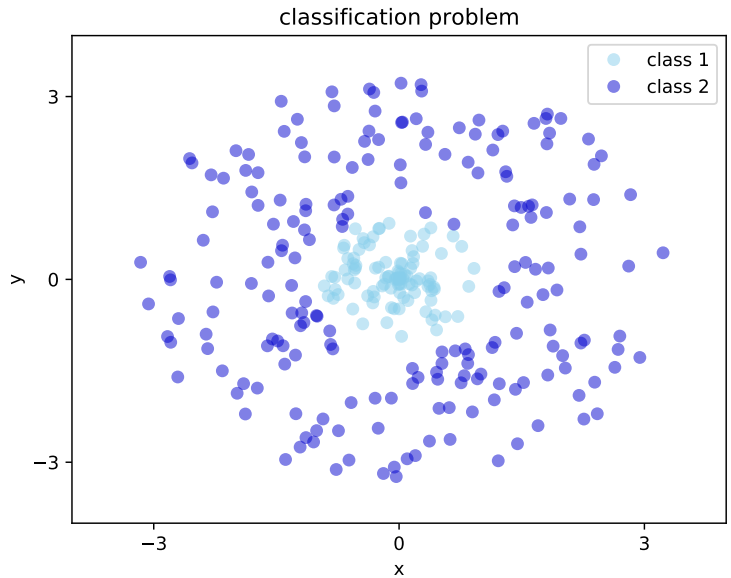


Figure

File : **code/-day_2/1_logistic_regression/feature_maps/vanilla_lr.py**.

Feature maps

Exercice 4 : Could we **transform the data** and then apply a logistic regression on them in order to have good train and test accuracies ?



Use **feature_maps_lr.py** in order to transform the data and correctly classify them with logistic regression.